

Architecture And Design Notes

This project is a .NET 9 Web API for Conway's Game of Life. It provides API endpoints, persistent board storage, tests, Swagger for Development, and a small browser UI for demonstration.

High-Level Architecture

The application is organized into a few focused areas:

```
GameOfLife.Api/  
  Models/  
    BoardRequests.cs  
  Services/  
    BoardValidator.cs  
    BoardStorageOptions.cs  
    GameOfLifeEngine.cs  
    IBoardStore.cs  
    JsonBoardStore.cs  
    StoredBoard.cs  
  wwwroot/  
    index.html  
  Program.cs  
  
GameOfLife.Tests/  
  UnitTest1.cs
```

`Program.cs` is the API composition layer. It registers dependencies, configures Swagger only for Development, serves the static demo UI, and maps the HTTP endpoints.

`Models/` contains the request and response DTOs used by the API.

`Services/` contains the application logic:

- `GameOfLifeEngine` handles Conway's Game of Life rules.
- `BoardValidator` validates and normalizes board input.
- `IBoardStore` defines the persistence contract.
- `JsonBoardStore` persists boards to disk as JSON.
- `BoardStorageOptions` holds persistence configuration.

`wwwroot/index.html` is a lightweight static UI. It calls the API endpoints using JavaScript.

`GameOfLife.Tests` contains unit and integration tests proving the rules, persistence, API behavior, and UI serving.

API Endpoints

The main endpoints are:

```
POST /boards
GET  /boards/{id}
GET  /boards/{id}/next
GET  /boards/{id}/states/{generations}
GET  /boards/{id}/final?maxAttempts=1000
```

`POST /boards` uploads a new board and returns the board id.

`GET /boards/{id}` returns the original uploaded board.

`GET /boards/{id}/next` returns the next generation from the original uploaded board.

`GET /boards/{id}/states/{generations}` returns the board after a requested number of generations.

`GET /boards/{id}/final` attempts to find the final stable state. If the board does not stabilize within `maxAttempts`, the API returns an error.

Request And Response Format

The API accepts either a readable row format:

```
{
  "rows": [
    ".....",
    "..#..",
    "..#..",
    "..#..",
    "....."
  ]
}
```

Or a boolean matrix:

```
{
  "cells": [
    [false, false, false],
    [false, true, false],
    [false, false, false]
  ]
}
```

The API returns board states with both formats:

```
{
  "id": "00000000-0000-0000-0000-000000000000",
  "generation": 1,
  "cells": [
    [false, false, false],
    [true, true, true],
    [false, false, false]
  ],
  "isStable": false,
  "rows": [
    "...",
    "###",
    "..."
  ]
}
```

Upload Flow

```
Client / Swagger / Demo UI
    |
    v
POST /boards
    |
    v
UploadBoardRequest
    |
    v
BoardValidator.TryNormalize()
    |
    |-- accepts rows: "." and "#"
    |-- accepts cells: bool[][]
    |-- validates rectangular shape
    v
JsonBoardStore.CreateAsync()
    |
    v
GameOfLife.Api/Data/boards.json
    |
    v
BoardCreatedResponse { id }
```

The upload endpoint does not calculate a new state. It only validates, normalizes, stores the board, and returns an id.

Next State Flow

```

Client / Swagger / Demo UI
    |
    v
GET /boards/{id}/next
    |
    v
JsonBoardStore.GetAsync(id)
    |
    v
GameOfLifeEngine.Next(cells)
    |
    v
BoardStateResponse

```

`/next` calculates one generation from the original uploaded board.

For a blinker pattern:

```

.....
..#..
..#..
..#..
.....

```

The next state becomes:

```

.....
.....
.###.
.....
.....

```

N Generations Flow

```

Client / Swagger / Demo UI
    |
    v
GET /boards/{id}/states/{generations}
    |
    v
JsonBoardStore.GetAsync(id)
    |
    v
GameOfLifeEngine.Advance(cells, generations)
    |
    |-- repeatedly calls Next()

```

v

BoardStateResponse

This is the endpoint to use when you want recursive generation behavior.

For example, a blinker becomes horizontal after one generation and vertical again after two generations:

Generation 0:

```
.....  
..#..  
..#..  
..#..  
.....
```

Generation 1:

```
.....  
.....  
.###.  
.....  
.....
```

Generation 2:

```
.....  
..#..  
..#..  
..#..  
.....
```

Final State Flow

Client / Swagger / Demo UI

|

v

GET /boards/{id}/final?maxAttempts=1000

|

v

JsonBoardStore.GetAsync(id)

|

v

GameOfLifeEngine.TryFindFinalState(cells, maxAttempts)

|

|-- calculate next generation

|-- compare current and next state

|-- stop if stable

|-- return error if maxAttempts is reached

v

BoardStateResponse or ErrorResponse

A final state means the next generation is identical to the current generation. Oscillating boards, like a blinker, do not become final because they keep changing.

Persistence

Boards are stored in:

```
GameOfLife.Api/Data/boards.json
```

This lets the API restart and still retain uploaded boards. The `Data` folder is ignored by git because it is runtime data.

The store writes to a temporary file first and then replaces the real file. This reduces the chance of corrupting the stored boards during a write.

Demo UI Flow

```
Browser opens /
  |
  v
wwwroot/index.html loads
  |
  v
User enters rows using "." and "#"
  |
  v
Upload Board button calls POST /boards
  |
  v
UI stores returned board id
  |
  v
Buttons call:
- GET /boards/{id}
- GET /boards/{id}/next
- GET /boards/{id}/states/{generations}
- GET /boards/{id}/final?maxAttempts={value}
  |
  v
UI renders visual grid and JSON response
```

The UI is intentionally static and simple. It demonstrates the API without adding a separate frontend framework or build step.

SOLID Principles

Single Responsibility Principle

Each major class has one primary responsibility:

- `GameOfLifeEngine` calculates board states.
- `BoardValidator` validates and parses board inputs.
- `JsonBoardStore` handles JSON file persistence.
- `Program.cs` wires HTTP endpoints to services.
- `BoardStateResponse` represents output data.

This keeps the code easier to test and reason about.

Open/Closed Principle

The persistence layer is behind `IBoardStore`. The API can be extended with another storage option, such as SQLite, Postgres, or cloud storage, without rewriting the endpoint logic.

For example, a future `SqlBoardStore` could implement `IBoardStore` and be registered in dependency injection.

Liskov Substitution Principle

The endpoints depend on the `IBoardStore` interface. Any implementation of `IBoardStore` should be usable by the API as long as it honors the same contract:

```
CreateAsync(...)  
GetAsync(...)
```

This allows `JsonBoardStore` to be replaced without changing the endpoint behavior.

Interface Segregation Principle

`IBoardStore` is intentionally small. It only exposes the operations the API needs:

```
Task<Guid> CreateAsync(bool[][] cells, CancellationToken cancellationToken);  
Task<StoredBoard?> GetAsync(Guid id, CancellationToken cancellationToken);
```

It does not force unrelated methods like update, delete, search, or pagination.

Dependency Inversion Principle

The API does not manually construct its dependencies inside each endpoint. Dependencies are registered with .NET dependency injection:


```
builder.Services.AddSingleton<GameOfLifeEngine>();  
builder.Services.AddSingleton<IBoardStore, JsonBoardStore>();
```

Then the endpoints receive services as parameters. This keeps the API layer loosely coupled and easier to test.

Completeness

The project includes tests for:

- Conway's Game of Life rule behavior.
- Stable final state detection.
- JSON persistence across store instances.
- API upload behavior.
- Next-state behavior.
- Readable `rows` response format.
- Final-state error behavior.
- Demo UI serving from `/`.

Run all tests with:

```
dotnet test
```

The expected result is that all tests pass